

Пермский институт (филиал) федерального государственного бюджетного
образовательного учреждения высшего образования

«Российский экономический университет имени Г. В. Плеханова»

Кафедра информационных технологий и программирования

Практическая работа

Тема работы: Создание системы управления складом

Выполнил: _____Харин Глеб

Алексеевич_____

Группа: _____ИПс-31_____

Пермь 2026

Оглавление

Введение	5
1. Анализ предметной области	6
1.1. Основные понятия	6
1.2. Сценарии использования	6
1.3. Требования к системе	6
2. Проектирование архитектуры приложения	8
2.1. Паттерн MVVM	8
2.2. Структура решения	8
2.3. Слои приложения	9
3. Проектирование модели данных	10
3.1. Organization	10
3.2. Warehouse	10
3.3. Product	10
3.4. Invoice и InvoiceItem	11
3.5. Схема связей	12
3.6. Хранилище DataStore	13
4. Реализация сервисного слоя	14
4.1. Интерфейс IDataService<T>	14
4.2. OrganizationService	14
4.3. WarehouseService	15
4.4. ProductService	15
4.5. InvoiceService	16
4.6. Формат CSV	18
5. Реализация пользовательского интерфейса (WPF)	19
5.1. Тема оформления	19
5.2. ViewModelBase и RelayCommand	19
5.3. Навигация	19
5.4. Главное окно	20
5.5. Диалог организации	20
5.6. Диалог подтверждения удаления	21
5.7. Окно складов и товаров	21

5.8. Диалоги склада и товара	22
5.9. Окно накладных	23
5.10. Диалог создания накладной	24
5.11. Просмотр накладной	26
5.12. Валидация и обработка ошибок	26
6. Тестирование приложения	27
6.1. Настройка	27
6.2. Состав тестов	27
6.3. Прогон	28
7. Ручное тестирование и демонстрация работы	29
7.1. Запуск	29
7.2. Организации	29
7.3. Склады	29
7.4. Товары	30
7.5. Импорт CSV	30
7.6. Поиск	31
7.7. Накладные	31
Заключение	34
Список использованных источников	35
Приложение А. Листинги ключевых классов	36
Приложение Б. Пример CSV-файла для импорта	38

Введение

Цель работы - разработать WPF-приложение «Управление складом», обеспечивающее ведение справочных данных об организациях, складах и товарах, а также учёт движения товаров с помощью накладных.

Для достижения цели поставлены задачи:

1. Проанализировать предметную область и выделить ключевые сущности.
2. Спроектировать архитектуру приложения в соответствии с паттерном MVVM.
3. Реализовать модели данных и сервисы CRUD-операций.
4. Реализовать пользовательский интерфейс на WPF.
5. Реализовать бизнес-логику накладных с изменением остатков товаров.
6. Покрыть сервисы модульными тестами.
7. Разместить проект на GitHub.

Технологический стек: язык C#, платформа .NET 8.0, WPF, архитектурный паттерн MVVM, тестовый фреймворк xUnit. Данные хранятся в коллекциях в оперативной памяти - база данных в работе не используется.

1. Анализ предметной области

1.1. Основные понятия

- Организация - юридическое лицо, владеющее одним или несколькими складами.
- Склад - место хранения товаров, привязанное к конкретной организации.
- Товар - единица складского учёта с набором характеристик: артикул, наименование, категория, производитель, поставщик, цена, остаток, скидка.
- Накладная - документ, фиксирующий партию товаров, перемещаемых на склад или со склада. Бывает входящей и исходящей.

1.2. Сценарии использования

1. Пользователь выбирает организацию из списка.
2. Внутри организации работает со списком её складов.
3. Внутри склада работает со списком товаров.
4. Оформляет накладные, проведение которых изменяет остатки товаров.

1.3. Требования к системе

- CRUD-операции над организациями, складами, товарами и накладными.
- Удаление организации с подтверждением через ввод её наименования.
- Каскадное удаление: организация → склады → товары.
- Просмотр товаров в таблице с колонками: фото, наименование, категория, производитель, поставщик, цена, остаток, скидка.
- Импорт товаров из CSV.

- Проведение и отмена проведения накладных с изменением остатков.

- Запрет отрицательного остатка.

- Поиск товаров по наименованию и артикулу.

Нефункциональные:

- Единая тема оформления.

- Наглядное отображение текущего контекста (организация, склад).

- Валидация пользовательского ввода.

- Покрытие сервисов модульными тестами.

2. Проектирование архитектуры приложения

2.1. Паттерн MVVM

Приложение построено на паттерне Model-View-ViewModel, который разделяет ответственность между:

- Model - модели данных и сервисы (WarehouseManager.Domain).
- ViewModel - состояние экрана и команды (WarehouseManager.App/ViewModels).
- View - XAML-описания окон (WarehouseManager.App/Views).

Паттерн обеспечивает слабую связанность слоёв и упрощает тестирование: сервисный слой можно проверять независимо от UI.

2.2. Структура решения

Решение "WarehouseManager" (3 проекта)

- └─ WarehouseManager.App - WPF-приложение
- └─ WarehouseManager.Domain - библиотека классов (модели + сервисы)
- └─ WarehouseManager.Tests - модульные тесты

WarehouseManager.App ссылается на WarehouseManager.Domain;
WarehouseManager.Tests ссылается на WarehouseManager.Domain.

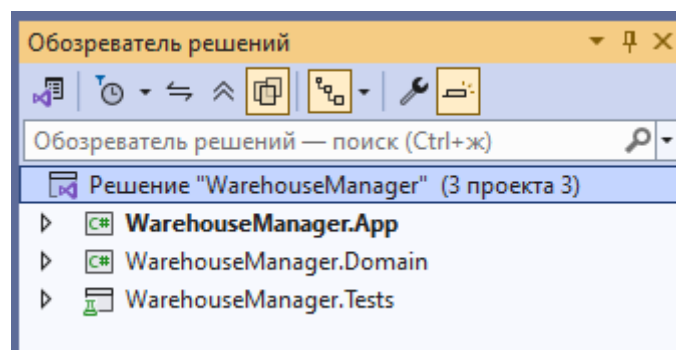


Рисунок 1 - Дерево решения в обозревателе Visual Studio

2.3. Слои приложения

Слой	Проект	Содержимое
Модели	Domain	Organization, Warehouse, Product, Invoice, InvoiceItem
Сервисы	Domain	OrganizationService, WarehouseService, ProductService, InvoiceService, DataStore
ViewModel	App	MainViewModel, WarehousesViewModel, InvoicesViewModel и др.
View	App	MainWindow, WarehousesWindow, диалоговые окна
Команды	App	RelayCommand
Тесты	Tests	ServicesTests

3. Проектирование модели данных

Все модели размещены в проекте WarehouseManager.Domain в папке Models.

3.1. Organization

Содержит идентификатор и наименование. Наименование не может быть пустым.

```
public class Organization
{
    public int Id { get; set; }
    public string Name { get; set; } = string.Empty;
}
```

3.2. Warehouse

Привязан к организации через OrganizationId, имеет наименование и адрес.

```
public class Warehouse
{
    public int Id { get; set; }
    public int OrganizationId { get; set; }
    public string Name { get; set; } = string.Empty;
    public string Address { get; set; } = string.Empty;
}
```

3.3. Product

Содержит все атрибуты товара, требуемые заданием.

```
public class Product
```

```

{

    public int Id { get; set; }

    public int WarehouseId { get; set; }

    public string Article { get; set; }

    public string Name { get; set; }

    public string Category { get; set; }

    public string Manufacturer { get; set; }

    public string Supplier { get; set; }

    public decimal Price { get; set; }

    public int Stock { get; set; }

    public double DiscountPercent { get; set; }

    public string Description { get; set; }

    public string ImagePath { get; set; }

}

```

Поле ImagePath используется для колонки «Фото»: в таблице отображается изображение по указанному пути.

3.4. Invoice и InvoiceItem

Накладная содержит список позиций и общий признак типа.

```

public enum InvoiceType { Incoming, Outgoing }

public enum InvoiceStatus { Draft, Posted }

public class Invoice

{

```

```

    public int Id { get; set; }

    public string Number { get; set; }

    public DateTime Date { get; set; }

    public InvoiceType Type { get; set; }

    public InvoiceStatus Status { get; set; }

    public int WarehouseId { get; set; }

    public string Comment { get; set; }

    public List<InvoiceItem> Items { get; set; } = new();

    public decimal Total => Items.Sum(i => i.Total);
}

public class InvoiceItem
{
    public int ProductId { get; set; }

    public string Article { get; set; }

    public string ProductName { get; set; }

    public int Quantity { get; set; }

    public decimal Price { get; set; }

    public decimal Total => Quantity * Price;
}

```

3.5. Схема связей

- Организация 1 → N Складов.
- Склад 1 → N Товаров.
- Склад 1 → N Накладных.

- Накладная 1 → N Позиций (InvoiceItem).
- Позиция ссылается на товар по ProductId.

3.6. Хранилище DataStore

DataStore - singleton-класс, содержащий четыре ObservableCollection. В нём же - счётчики идентификаторов и метод Seed(), заполняющий коллекции тестовыми данными.

```
public class DataStore
{
    public ObservableCollection<Organization> Organizations { get; } =
new();

    public ObservableCollection<Warehouse> Warehouses { get; } = new();

    public ObservableCollection<Product> Products { get; } = new();

    public ObservableCollection<Invoice> Invoices { get; } = new();


    public void Seed() { /* заполнение тестовыми данными */ }

    public void Clear() { /* очистка всех коллекций */ }

}
```

Тестовые данные: две организации («ООО Ромашка», «ИП Иванов»), три склада, четыре товара в разных категориях. Этого достаточно для демонстрации всех сценариев.

4. Реализация сервисного слоя

Все сервисы расположены в WarehouseManager.Domain/Services.

4.1. Интерфейс IDataService<T>

Единый контракт CRUD-сервисов:

```
public interface IDataService<T> where T : class
```

```
{
```

```
    IReadOnlyList<T> GetAll();
```

```
    T? GetById(int id);
```

```
    void Add(T item);
```

```
    void Update(T item);
```

```
    void Delete(int id);
```

```
}
```

Единый интерфейс обеспечивает однотипность работы со всеми сущностями.

4.2. OrganizationService

Реализует CRUD организаций. При добавлении и обновлении проверяет, что Name не пустое. При удалении выполняет каскадное удаление: сначала удаляются товары всех складов организации, затем сами склады, затем организация.

```
public void Delete(int id)
```

```
{
```

```
    var org = GetById(id);
```

```
    if (org is null) return;
```

```

var warehouseIds = _store.Warehouses

.Where(w => w.OrganizationId == id)

.Select(w => w.Id)

.ToList();

foreach (var p in _store.Products.Where(p =>
warehouseIds.Contains(p.WarehouseId)).ToList())

    _store.Products.Remove(p);

foreach (var w in _store.Warehouses.Where(w => w.OrganizationId ==
id).ToList())

    _store.Warehouses.Remove(w);

_store.Organizations.Remove(org);
}

```

4.3. WarehouseService

Помимо CRUD содержит метод `GetByOrganization(int organizationId)`, возвращающий склады конкретной организации. При добавлении склада проверяется существование родительской организации. При удалении склада удаляются все его товары.

4.4. ProductService

Ключевые методы:

- `GetByWarehouse(int warehouseId)` - выборка товаров склада.
- `ChangeStock(int productId, int delta)` - изменение остатка на величину `delta`. Если итог отрицательный, выбрасывается `InvalidOperationException`.

- `ImportFromCsv(string path, int warehouseId)` - импорт товаров из файла.

Валидация в методе `Validate`:

- артикул и наименование не пустые;
- $\text{цена} \geq 0$;
- $\text{остаток} \geq 0$;
- скидка в диапазоне 0–100 %.

4.5. InvoiceService

Сервис накладных работает поверх `ProductService` - это позволяет централизованно менять остатки и не дублировать логику.

Ключевые методы:

- `Add(Invoice)` - создаёт черновик; проверяет, что в накладной хотя бы одна позиция.
- `Update(Invoice)` - запрещено для проведённых накладных.
- `Delete(int)` - запрещено для проведённых накладных.
- `Post(int)` - проводит накладную.
- `Unpost(int)` - отменяет проведение.

Проведение входящей накладной увеличивает остаток каждой позиции; проведение исходящей - уменьшает. Перед уменьшением проверяется достаточность остатка.

```
public void Post(int invoiceId)

{

var inv = GetById(invoiceId)

?? throw new InvalidOperationException("Накладная не найдена.");
```

```
if (inv.Status == InvoiceStatus.Posted)

throw new InvalidOperationException("Накладная уже проведена.");


if (inv.Type == InvoiceType.Outgoing)

{

foreach (var item in inv.Items)

{

var product = _productService.GetById(item.ProductId)

?? throw new InvalidOperationException($"Товар {item.Article} не
найден.");

if (product.Stock < item.Quantity)

throw new InvalidOperationException(

$"Недостаточно товара «{product.Name}» на складе.");

}

}


foreach (var item in inv.Items)

{

var delta = inv.Type == InvoiceType.Incoming ? item.Quantity : -
item.Quantity;

_productService.ChangeStock(item.ProductId, delta);

}
```

```
inv.Status = InvoiceStatus.Posted;  
  
}
```

Отмена проведения - обратная операция.

4.6. Формат CSV

Файл имеет разделитель ; и необязательную строку заголовка. Колонки:

Article;Name;Category;Manufacturer;Supplier;Price;Stock;Discount

Пример:

IMP-001;Импортный товар 1;Электроника;Apple;ООО
ТехноТрейд;15000;5;0

IMP-002;Импортный товар 2;Мебель;IKEA;ООО МебельСнаб;8000;3;5

5. Реализация пользовательского интерфейса (WPF)

5.1. Тема оформления

Тема вынесена в Resources/Theme.xaml и подключена в App.xaml через MergedDictionaries. Используется тёмный фон, акцентный цвет #3FA7D6, отдельные стили для Button, TextBox, DataGrid, ListBox, Menu.

```
<Application.Resources>

  <ResourceDictionary>

    <ResourceDictionary.MergedDictionaries>

      <ResourceDictionary Source="Resources/Theme.xaml"/>

    </ResourceDictionary.MergedDictionaries>

  </ResourceDictionary>

</Application.Resources>
```

5.2. ViewModelBase и RelayCommand

ViewModelBase реализует INotifyPropertyChanged и предоставляет методы OnPropertyChanged и Set. Все ViewModel наследуются от него.

RelayCommand реализует ICommand и принимает два делегата: execute и опциональный canExecute. Это позволяет автоматически блокировать недоступные команды: например, кнопка «Изменить» неактивна, пока не выбран элемент.

5.3. Навигация

Навигация построена по схеме «контекст → вложенный контекст»:

1. MainWindow - выбор организации.
2. Кнопка ОК открывает WarehousesWindow для выбранной организации.
3. В WarehousesWindow слева - список складов, справа - таблица товаров.

4. Из меню «Накладные» открывается InvoicesWindow для текущего склада.

Текущий контекст всегда виден в статус-баре: «Организация: ... | склад: ...».

5.4. Главное окно

Содержит ListBox организаций, кнопки ОК и Закреть, а также контекстное меню с командами добавления, редактирования и удаления.

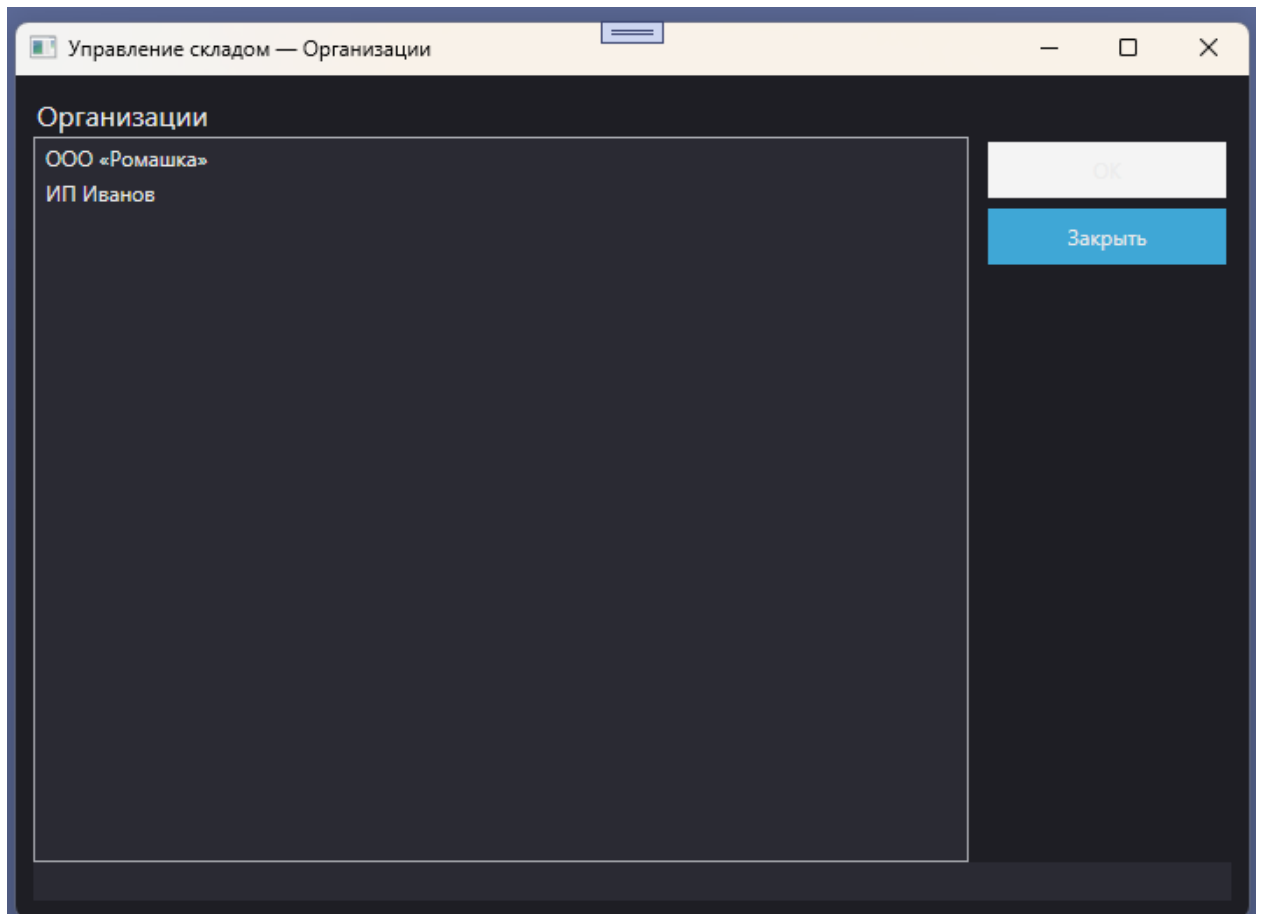


Рисунок 2 - Главное окно

5.5. Диалог организации

OrganizationDialogView - простое окно с одним полем. Перед закрытием проверяется, что поле не пустое.

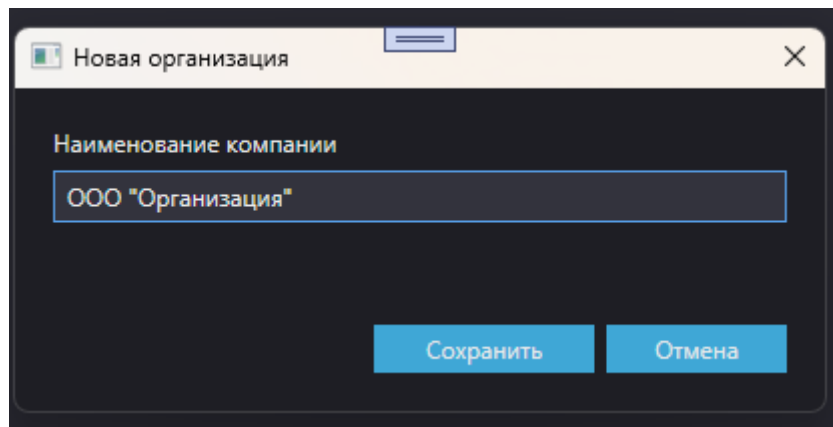


Рисунок 3 - Диалог создания организации

5.6. Диалог подтверждения удаления

ConfirmDeleteDialogView требует ввести точное название организации. Кнопка «Удалить» активируется только при совпадении с ExpectedName.

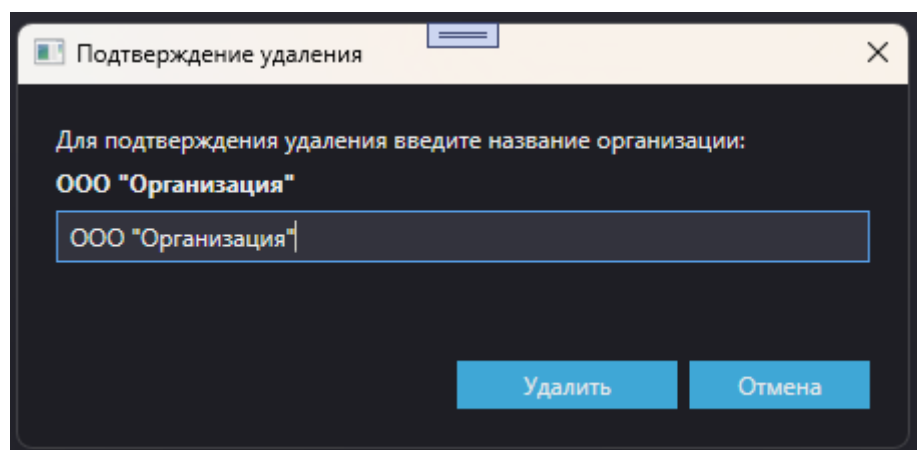


Рисунок 4 - Диалог подтверждения удаления

5.7. Окно складов и товаров

Основное рабочее окно. Слева - список складов выбранной организации, справа - таблица товаров с поиском. Верхнее меню содержит разделы Файл, Склады, Товары, Накладные.

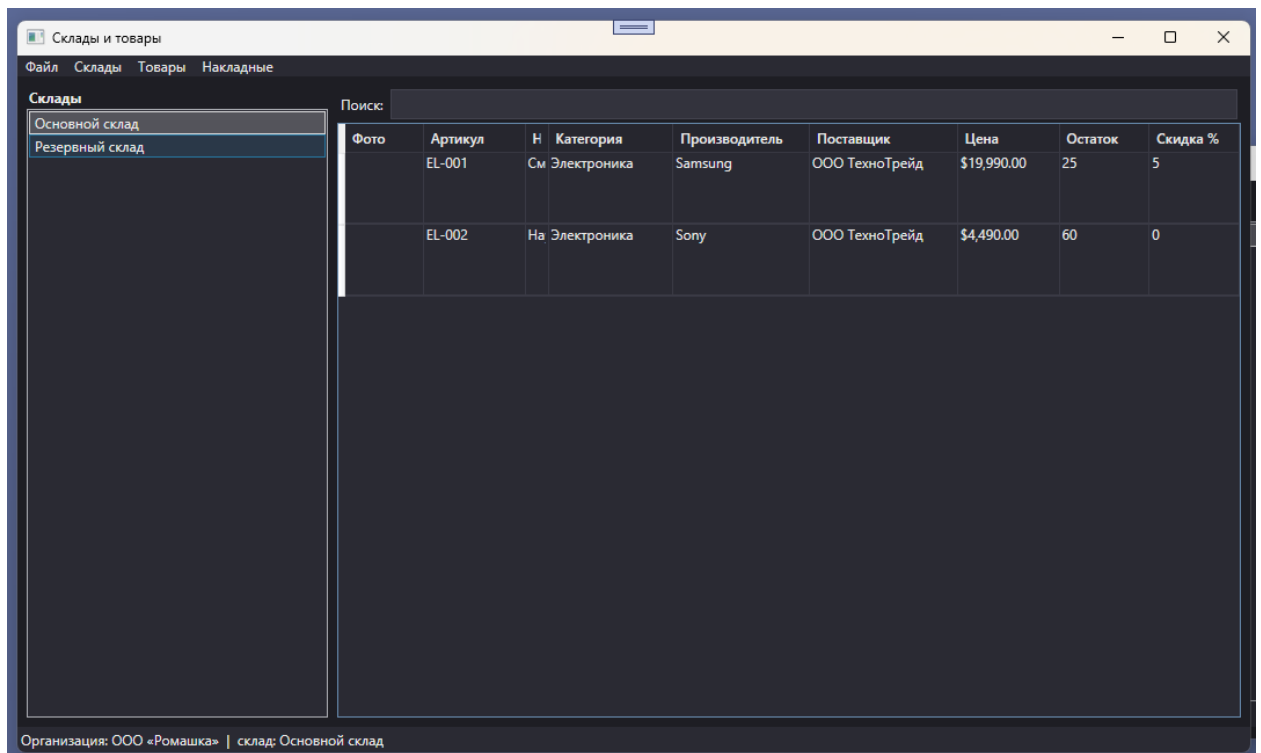


Рисунок 5 - Окно складов и товаров

5.8. Диалоги склада и товара

- WarehouseDialogView - поля «Наименование» и «Адрес».
- ProductDialogView - все поля товара, включая путь к фото.

В ProductDialogViewModel реализован метод TryBuild, который одновременно валидирует значения и собирает объект Product. Если хотя бы одно поле заполнено неверно, возвращается текст ошибки.

Артикул	EL-001
Наименование	Смартфон Galaxy A15
Категория	Электроника
Производитель	Samsung
Поставщик	ООО ТехноТрейд
Цена	19990
Остаток	25
Скидка, %	5
Путь к фото	
Описание	

Сохранить

Отмена

Рисунок 6 - Диалог товара

5.9. Окно накладных

InvoicesWindow отображает таблицу накладных выбранного склада. Столбцы: №, дата, тип, статус, комментарий. Кнопки: создать, просмотр, изменить, провести, отменить проведение, удалить. Кнопки «Изменить» и «Удалить» доступны только для черновиков.

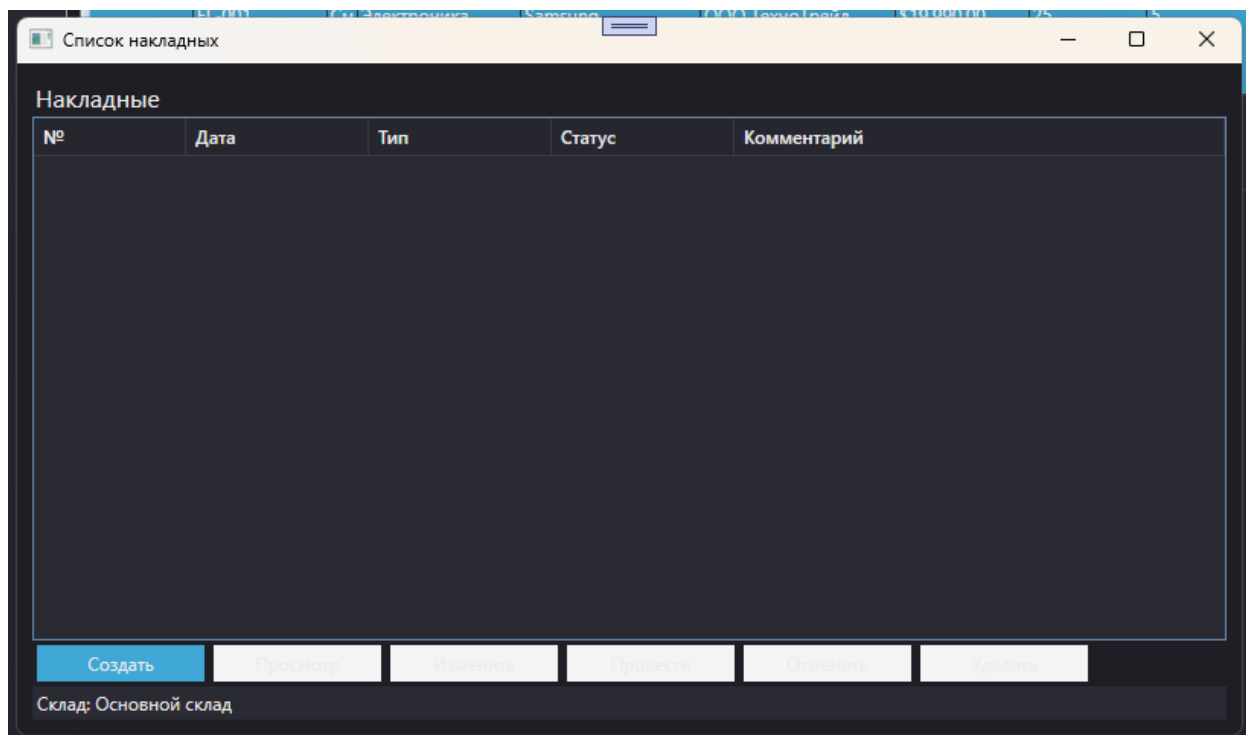


Рисунок 7 - Окно накладных

5.10. Диалог создания накладной

InvoiceDialogView позволяет выбрать тип, добавить комментарий и сформировать список позиций. Каждая позиция добавляется через InvoiceItemDialogView, где выбирается товар и указывается количество.

Новая накладная

Тип накладной

Комментарий

Товары в накладной

Артикул	Наименование	Кол-во	Цена	Сумма
---------	--------------	--------	------	-------

Добавить товар Удалить товар Сохранить Отмена

Рисунок 8 - Диалог создания накладной

Выбор товара

Выберите товар:

Артикул	Наименование	Остаток	Цена
EL-001	Смартфон Galaxy A15	25	\$19,990.00
EL-002	Наушники WH-CH520	60	\$4,490.00

Количество: 1 OK Отмена

Рисунок 9 - Выбор товара в накладную

5.11. Просмотр накладной

InvoiceDetailsView отображает шапку (номер, дата, тип, статус) и таблицу позиций.

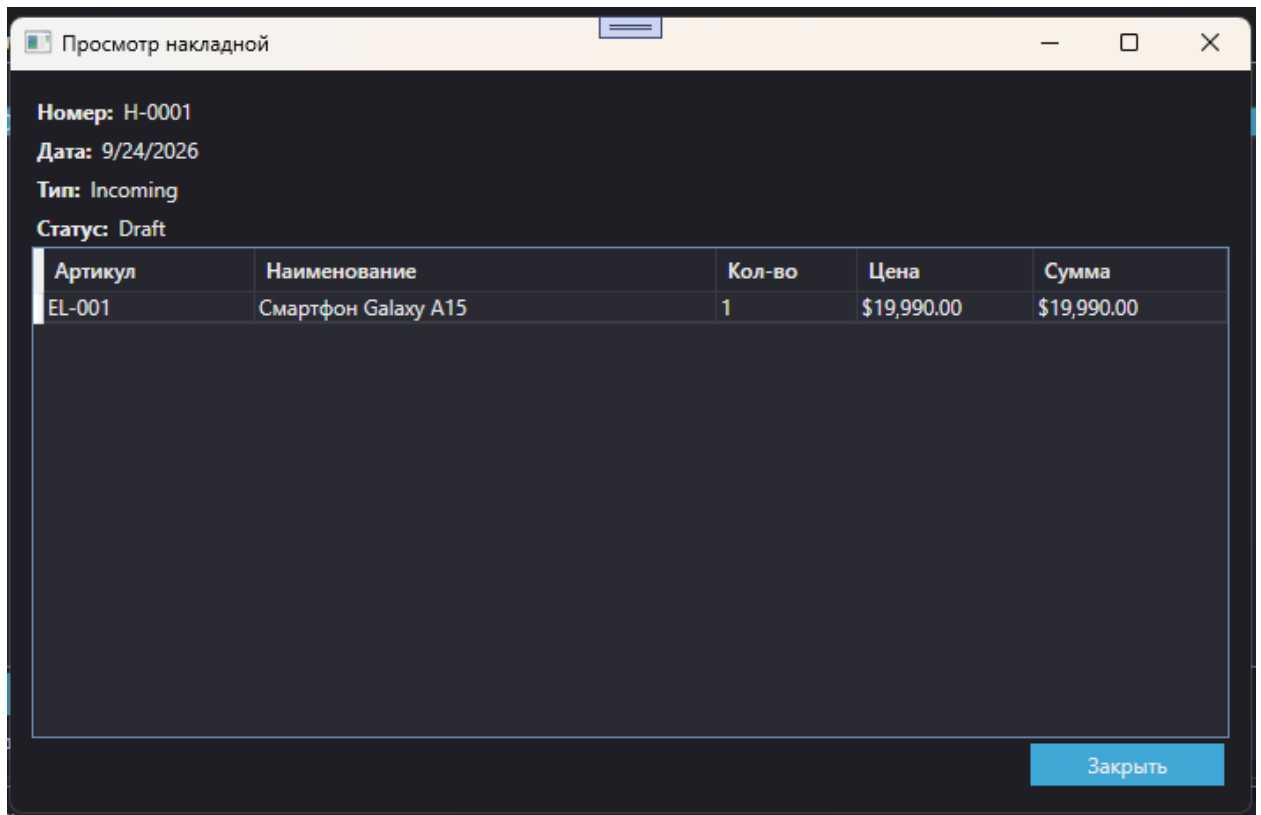


Рисунок 10 - Просмотр накладной

5.12. Валидация и обработка ошибок

- В диалогах проверка выполняется до закрытия окна.
- Ошибки сервисов (например, отрицательный остаток) перехватываются в ViewModel и выводятся через MessageBox.
- Поля, обязательные для заполнения, помечены в диалогах; в ProductDialogView проверяется числовой формат цены, остатка и скидки.

6. Тестирование приложения

6.1. Настройка

Тестовый проект создан по шаблону xUnit Test Project (.NET 8.0). В него добавлены пакеты:

- Microsoft.NET.Test.Sdk
- xunit
- xunit.runner.visualstudio

6.2. Состав тестов

Все тесты вынесены в один файл ServicesTests.cs. Используется приватный хелпер NewServices(), создающий изолированный DataStore и все сервисы для каждого теста.

Покрытие по группам:

OrganizationService: добавление, валидация пустого имени, поиск по Id, поиск по несуществующему Id, обновление, каскадное удаление.

WarehouseService: добавление с проверкой родительской организации, валидация пустого имени, фильтрация по организации, обновление, каскадное удаление товаров.

ProductService: валидация пустого артикула, отрицательной цены, отрицательного остатка, скидки вне диапазона; фильтрация по складу; изменение остатка вверх и вниз; запрет отрицательного остатка; обновление; удаление.

InvoiceService: запрет пустой накладной; проведение входящей (увеличение остатка); проведение исходящей (уменьшение остатка); ошибка при недостаточном остатке; отмена проведения; запрет редактирования и удаления проведённой накладной.

Всего - 25 тестов.

6.3. Прогон

Для запуска использовалось меню Тест → Запустить все тесты. Все 25 тестов завершились успешно.

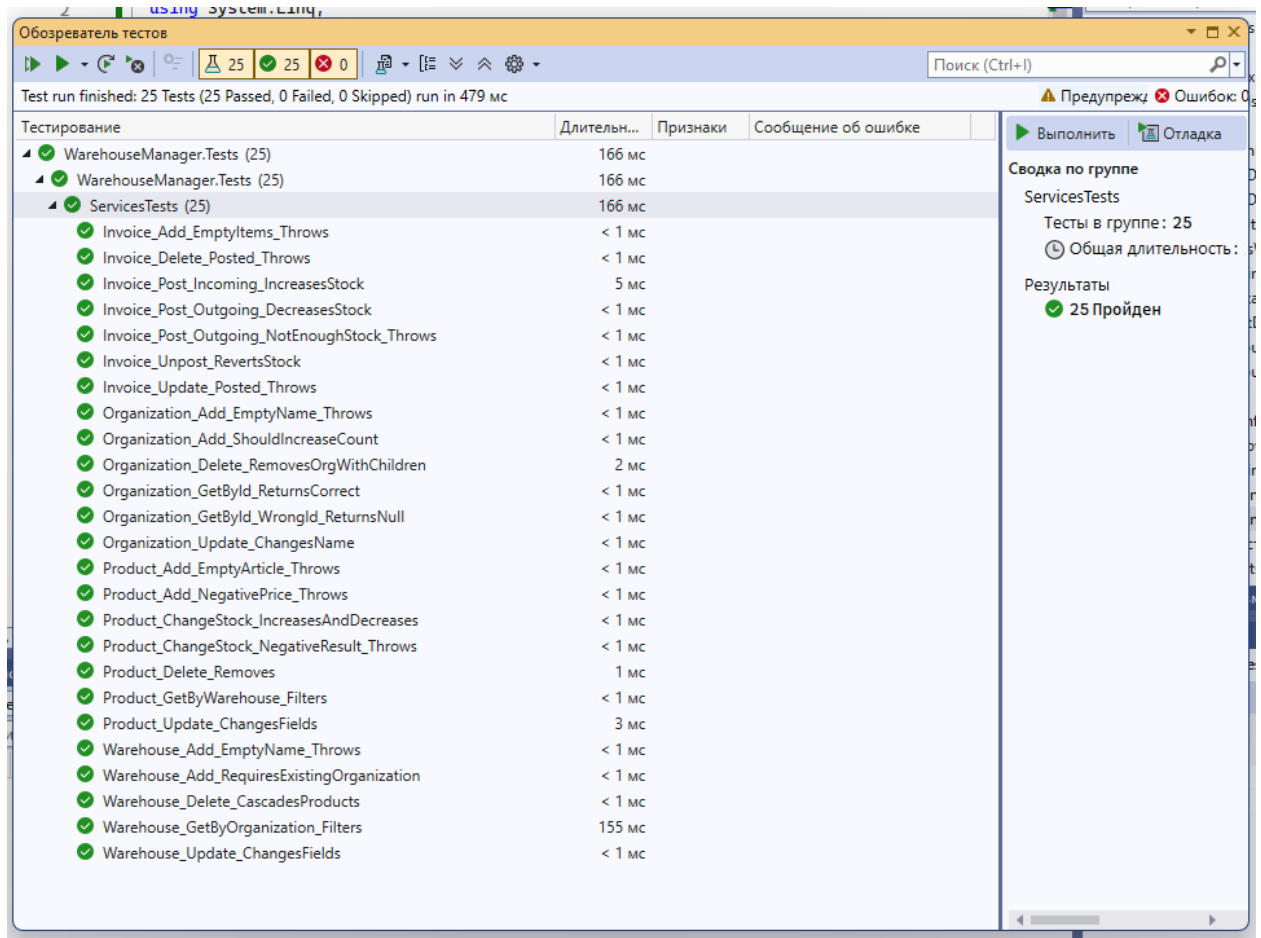


Рисунок 11 - Обозреватель тестов с зелёными результатами

7. Ручное тестирование и демонстрация работы

7.1. Запуск

Стартовый проект - WarehouseManager.App, запуск по F5.

7.2. Организации

После запуска открывается главное окно со списком организаций. Через контекстное меню добавлена новая организация; проверены редактирование и удаление с вводом названия.

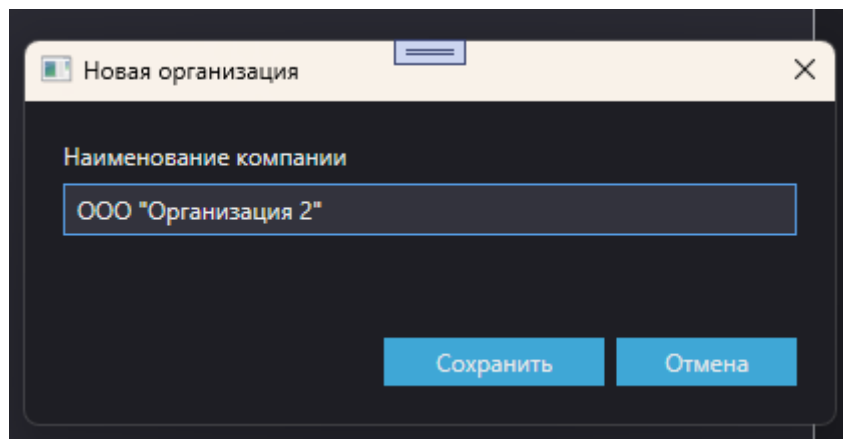


Рисунок 12 - Добавление новой организации

7.3. Склады

Выбрана организация «ООО Ромашка», нажата кнопка ОК. В открывшемся окне добавлен новый склад.

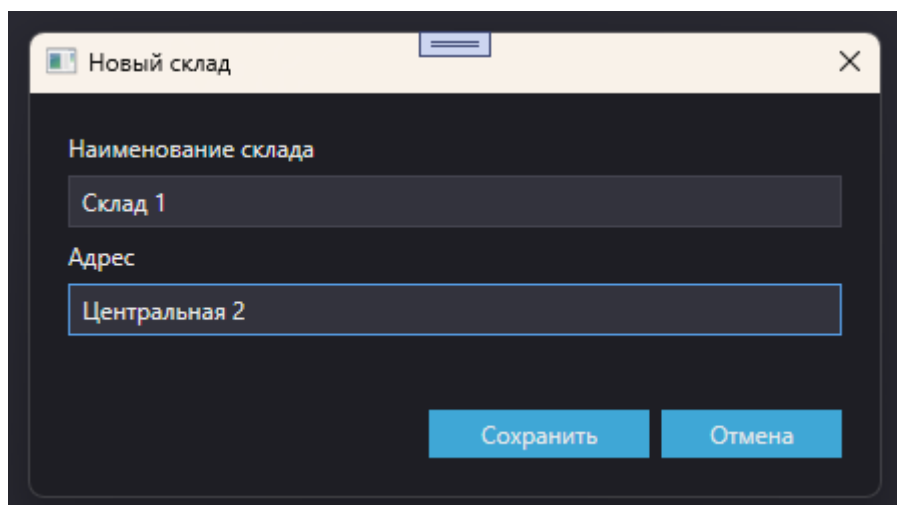


Рисунок 13 - Диалог создания склада

7.4. Товары

В новом складе добавлен товар с заполнением всех полей. Товар появился в таблице справа.

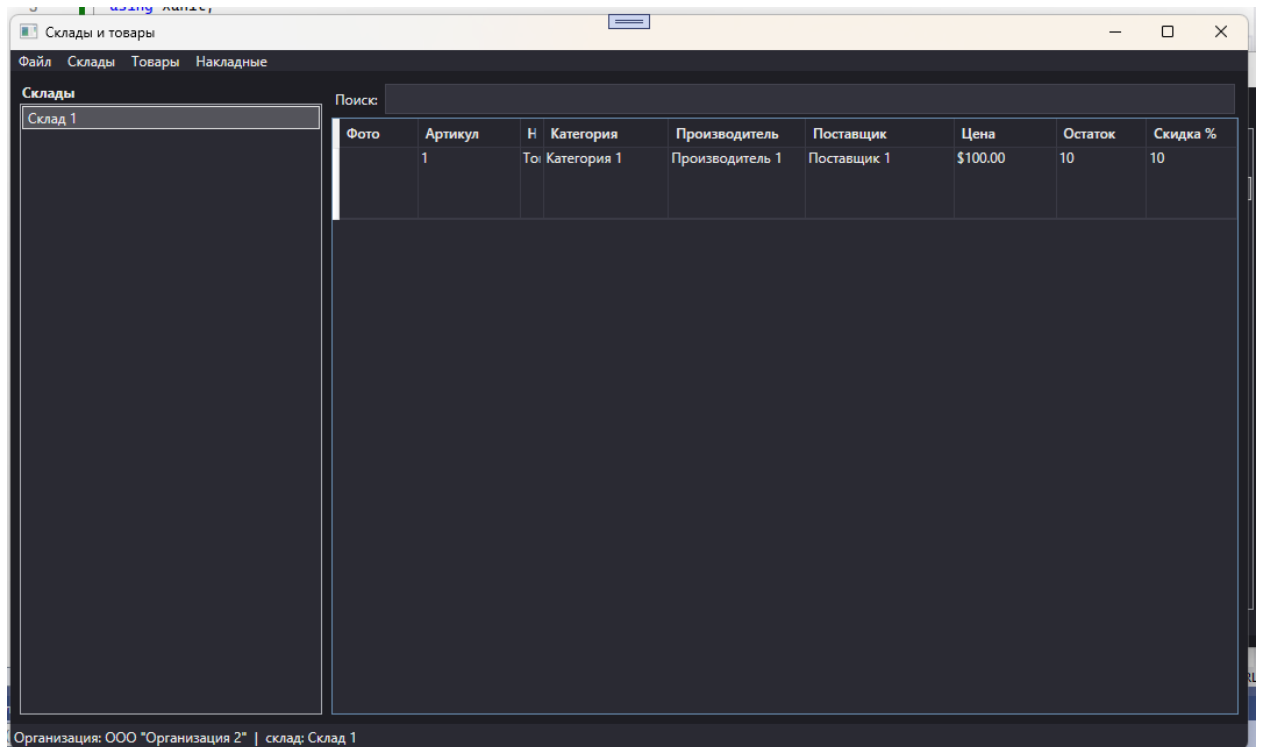


Рисунок 14 - Добавленный товар в таблице

7.5. Импорт CSV

Через меню Файл → Импорт товаров загружен файл из двух позиций. Оба товара появились в таблице; выведено сообщение «Импортировано товаров: 2».

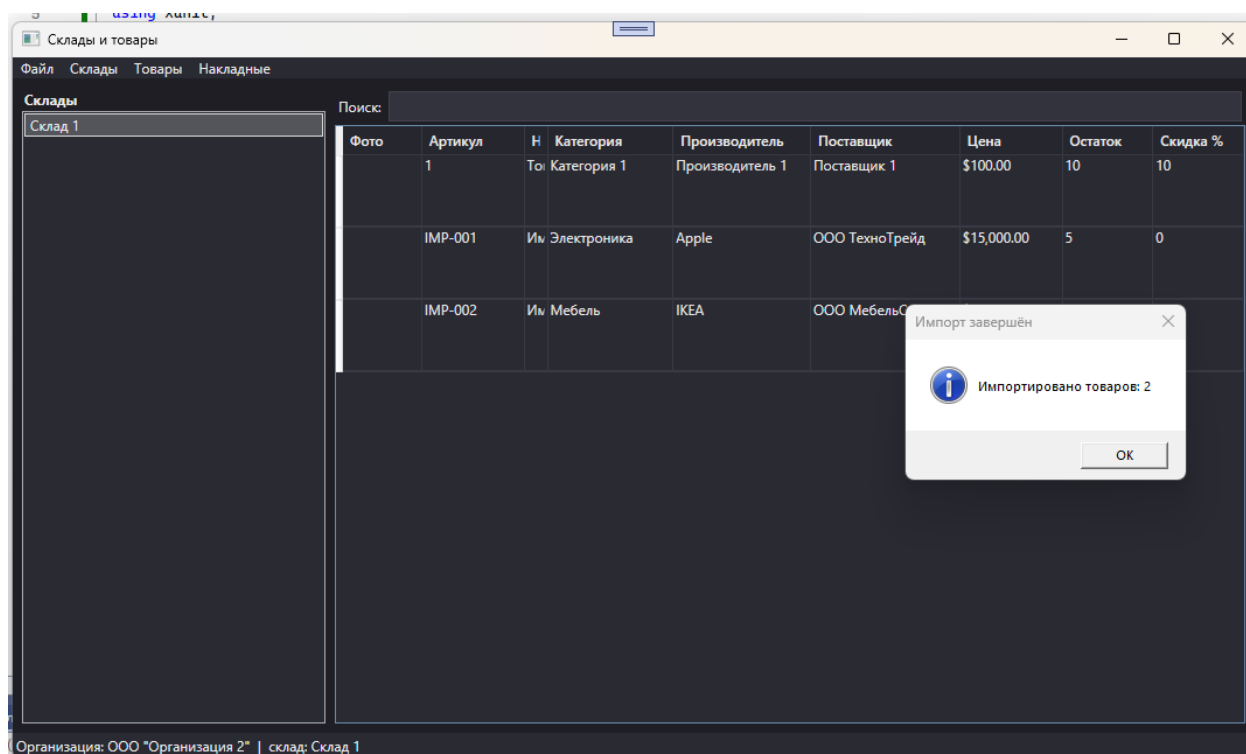


Рисунок 15 - Результат импорта CSV

7.6. Поиск

В строке поиска введён фрагмент названия - таблица отфильтрована.

7.7. Накладные

Через меню Накладные → Создать накладную создана входящая накладная на 5 единиц товара. После нажатия Провести остаток товара увеличился, что видно в таблице товаров.

Новая накладная

Тип накладной

Комментарий

Товары в накладной

Артикул	Наименование	Кол-во	Цена	Сумма
---------	--------------	--------	------	-------

Добавить товар

Удалить товар

Сохранить

Отмена

Рисунок 16 - Создание накладной

Склады и товары

Файл Склады Товары Накладные

Склады

Склад 1

Поиск:

Фото	Артикул	Н	Категория	Производитель	Поставщик	Цена	Остаток	Скидка %
	1	То	Категория 1	Производитель 1	Поставщик 1	\$100.00	11	10
	IMP-001	Им	Электроника	Apple	ООО ТехноТрейд	\$15,000.00	5	0
	IMP-002	Им	Мебель	IKEA	ООО МебельСнаб	\$8,000.00	3	5

Организация: ООО "Организация 2" | склад: Склад 1

Рисунок 17 - Изменённый остаток после проведения

Дополнительно проверены:

- проведение исходящей накладной на количество, превышающее остаток, - выведена ошибка;
- отмена проведения - остаток вернулся к исходному значению.

Заключение

В ходе работы разработано WPF-приложение «Управление складом», полностью соответствующее заданию.

Выполнено:

- три справочных модуля: организации, склады, товары;
- дополнительный модуль накладных с логикой проведения и отмены;
- корректное каскадное удаление связанных данных;
- валидация на уровне сервисов и на уровне интерфейса;
- импорт товаров из CSV;
- единая тёмная тема оформления;
- 25 модульных тестов, все проходят успешно.

Наиболее удачные решения:

- разделение решения на три проекта (Domain, App, Tests) облегчило тестирование;
- зависимость InvoiceService от ProductService позволила не дублировать логику изменения остатков;
- единый контракт IDataService<T> упростил реализацию однотипных CRUD-операций.

Возможные направления развития: подключение реальной базы данных (Entity Framework), поддержка ролей пользователей, добавление отчётов по движению товаров за период, расширение форматов импорта (Excel).

Список использованных источников

1. Microsoft. Documentation for WPF. - URL: <https://learn.microsoft.com/dotnet/desktop/wpf/>
2. Microsoft. Documentation for .NET 8. - URL: <https://learn.microsoft.com/dotnet/>
3. Microsoft. Model-View-ViewModel (MVVM) Pattern. - URL: <https://learn.microsoft.com/dotnet/architecture/maui/mvvm>
4. [xUnit.net](https://xunit.net/). Getting started. - URL: <https://xunit.net/>
5. Нейгел К., Ивсен Б., Глинн Дж., Уотсон К. С# 5.0 и платформа .NET 4.5 для профессионалов. - М.: Диалектика, 2013.

Приложение А. Листинги ключевых классов

```
public void Post(int invoiceId)

{

    var inv = GetById(invoiceId)

        ?? throw new InvalidOperationException("Накладная не найдена.");

    if (inv.Status == InvoiceStatus.Posted)

        throw new InvalidOperationException("Накладная уже проведена.");

    if (inv.Type == InvoiceType.Outgoing)

    {

        foreach (var item in inv.Items)

        {

            var product = _productService.GetById(item.ProductId)

                ?? throw new InvalidOperationException($"Товар {item.Article}

не найден.");

            if (product.Stock < item.Quantity)

                throw new InvalidOperationException(

                    $"Недостаточно товара «{product.Name}» на складе.");

        }

    }

    foreach (var item in inv.Items)

    {
```

```
        var delta = inv.Type == InvoiceType.Incoming ? item.Quantity : -  
item.Quantity;
```

```
        _productService.ChangeStock(item.ProductId, delta);  
    }
```

```
    inv.Status = InvoiceStatus.Posted;  
}
```

Приложение Б. Пример CSV-файла для импорта

Article;Name;Category;Manufacturer;Supplier;Price;Stock;Discount

IMP-001;Импортный товар 1;Электроника;Apple;ООО
ТехноТрейд;15000;5;0

IMP-002;Импортный товар 2;Мебель;IKEA;ООО МебельСнаб;8000;3;5